



Week10_예습과제_고은비

Chapter6. 차원 축소

- 1 개요
- 2 PCA (Principal Component Analysis)
- 3 LDA (Linear Discriminant Analysis)
- 4 SVD (Singular Value Decomposition)
- 5 NMF (Non-Negative Matrix Factorization)

📌 정리

Chapter6. 차원 축소

1 개요



차원 축소란?

매우 많은 피처로 구성된 다차원 데이터 세트의 차원을 축소해 새로운 차원의 데이터 세트를 생성하는 것

피처가 많을 경우

- 일반적으로 데이터 포인트 간의 거리가 기하급수적으로 멀어지게 되고 희소(sparse)한 구조를 가지게 됨
- 상대적으로 예측 신뢰도가 떨어짐
- 개별 피처간에 상관 관계가 높을 가능성이 커서 선형 모델에서의 다중 공선성 문제 발생 가능

⇒ 차원을 축소해 피처 수를 줄이면 더 직관적으로 데이터를 해석할 수 있다!

차원 축소

- 피처 선택 (Feature selection)
: 종속성이 강한 불필요한 피처를 제거, 주요 피처만 선택
- 피처 추출 (Feature extraction)
: 기존 피처를 중요 피처로 압축해서 추출 → 기존의 피처와는 완전히 다른 값을 가지게 됨
 - 단순 압축이 아닌, 함축적으로 설명 가능한 다른 공간으로 매핑하는 과정
 - 즉, 기존 피처가 전혀 인지하기 어려웠던 잠재적인 요소(Latent Factor)를 추출

차원 축소 알고리즘

PCA, SVD, NMF

→ 매우 많은 차원을 가지고 있는 이미지나 텍스트에서 차원 축소를 통해 잠재적인 의미를 찾아줌

- 매우 많은 픽셀로 이뤄진 이미지 데이터에서 잠재된 특성을 피처로 추출
 - 함축적 형태의 이미지 변환과 압축을 수행
 - 과적합 영향력이 작아져서 예측 성능이 좋아질 수 있음
- 문서 내 함축적 의미를 추출

- 문서 내 단어들의 구성에서 숨겨진 Semantic 의미나 Topic을 잠재 요소로 간주하고 이를 찾아낼 수 있음 (SVD, NMF)

2 PCA (Principal Component Analysis)

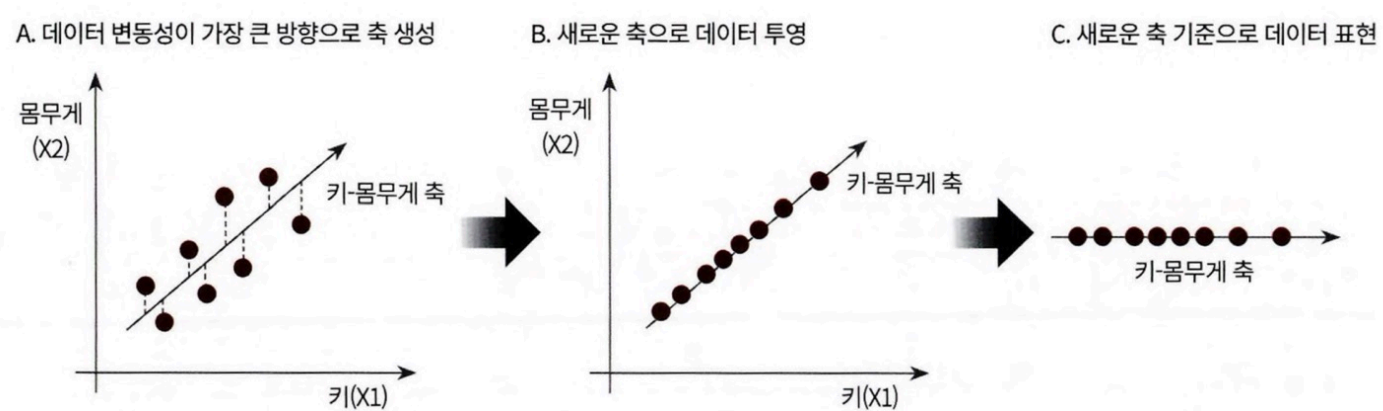


PCA

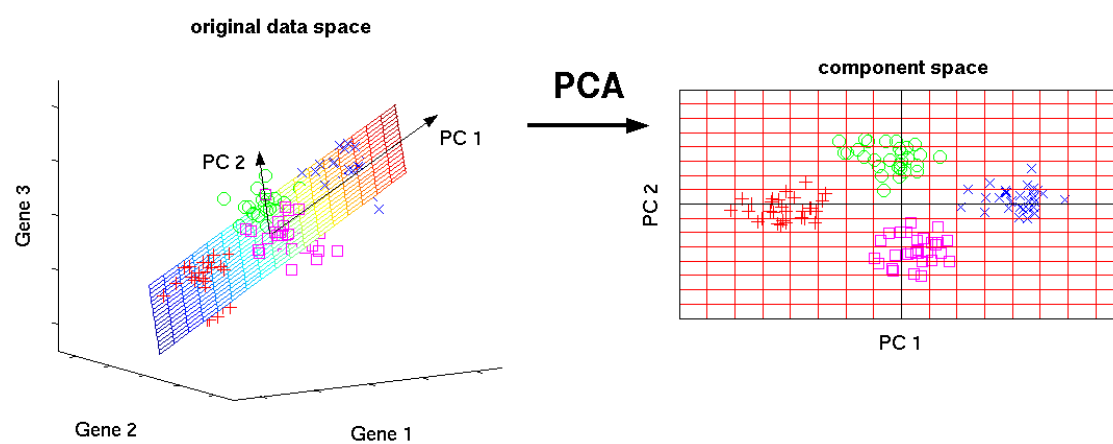
여러 변수 간에 존재하는 상관관계를 이용, 이를 대표하는 주성분(Principal Component)을 추출해 차원을 축소하는 가장 대표적인 차원 축소 기법

→ 데이터 정보 유실 최소화

개요



키, 몸무게 2개의 피처를 한개의 주성분을 가진 데이터 세트로 차원을 축소

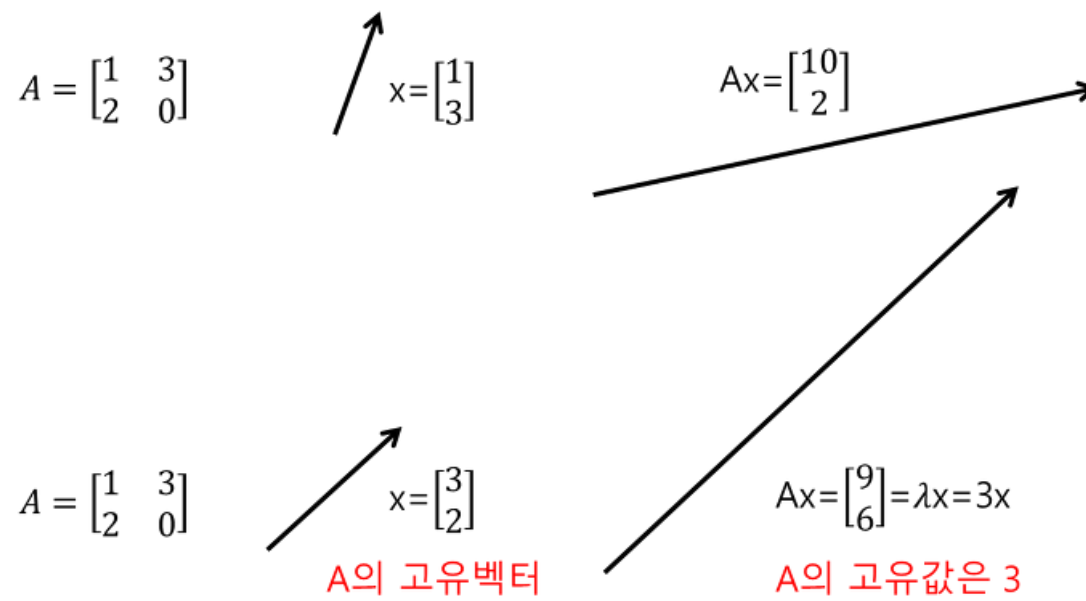


1. 먼저 가장 큰 데이터 변동성 (Variance)을 기반으로 첫번째 벡터 축을 생성
 2. 두 번째 축은 이 벡터 축에 직각이 되는 직교 벡터를 축으로 함
 3. 세 번째 축은 다시 두 번째 축과 직각이 되는 벡터를 설정하는 방식으로 축을 생성
- ⇒ 이렇게 생성된 벡터 축에 원본 데이터를 투영하면 축 개수만큼의 차원으로 원본 데이터가 축소됨

선형 대수 관점에서의 PCA

입력 데이터의 공분산 행렬(Covariance Matrix)을 고유값 분해하고

→ 고유 벡터에 입력 데이터를 선형 변환하는 것



- 고유 벡터 : 행렬 A를 곱하더라도 방향이 변하지 않고 크기만 변하는 벡터
 - 정방행렬은 차원 수 만큼의 고유 벡터를 가짐
⇒ 행렬을 분해하는 데 사용
 - PCA의 주성분 벡터, 입력 데이터의 분산이 큰 방향
- 고유값 (eigenvalue) : 고유 벡터의 크기 (=입력 데이터의 분산)
- 선형 변환 : 특정 벡터에 행렬 A를 곱해 새로운 벡터로 변환하는 것
= 특정 벡터를 하나의 공간에서 다른 공간으로 투영 (행렬 = 공간)
- 공분산 : 두 변수 간의 변동을 의미
 - $\text{Cov}(X, Y) > 0$ 이면 X가 증가할때 Y도 증가한다는 뜻
 - $\text{Cov}(X, X)$ 는 X의 분산
- 공분산 행렬

$$\begin{aligned}
 \text{Var}[\mathbf{x}] &= \begin{bmatrix} \text{Var}[x_1] & \text{Cov}[x_1, x_2] & \cdots & \text{Cov}[x_1, x_n] \\ \text{Cov}[x_2, x_1] & \text{Var}[x_2] & \cdots & \text{Cov}[x_2, x_n] \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}[x_n, x_1] & \text{Cov}[x_n, x_2] & \cdots & \text{Var}[x_n] \end{bmatrix} \\
 &= \begin{bmatrix} \sigma_{11} & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{21} & \sigma_{22} & \cdots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \cdots & \sigma_{nn} \end{bmatrix} = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \cdots & \sigma_n^2 \end{bmatrix}
 \end{aligned}$$

- Square + Symmetric 행렬
- 항상 고유 벡터를 직교 행렬 (orthogonal matrix)로, 고유값을 정방 행렬로 대각화 가능

$$C = P \Sigma P^T$$

C=공분산행렬, P=n*n 직교행렬, Σ=n*n 정방행렬

$$C = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$$

e=고유벡터(Ax=ax, 행렬A를 곱하더라도 방향이 변하지 않고 그 크기만 변하는 벡터), λ=고유벡터의 크기

⇒ 즉, 입력데이터의 공분산 행렬이 고유벡터 + 고유값으로 분해될 수 있으며, 이렇게 분해된 고유 벡터를 이용해 입력데이터를 선형 변환하는 방식이 PCA이다.

PCA 수행 단계

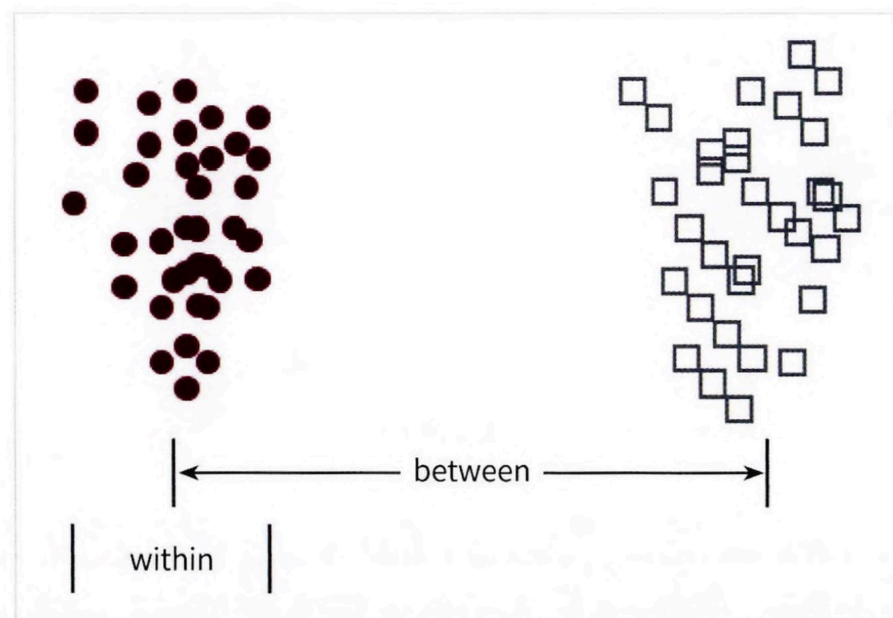
1. 입력 데이터 세트의 공분산 행렬을 생성
2. 공분산 행렬의 고유벡터와 고유값을 계산
3. 고유값이 가장 큰 순으로 K개 (=PCA 변환 차수) 만큼 고유벡터를 추출
4. 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

3 LDA (Linear Discriminant Analysis)



선형 판별 분석법

PCA와 매우 유사하게 입력데이터 세트를 저차원 공간에 투영에 차원을 축소하는 기법



- 클래스 분리를 최대화하는 축을 찾기 위해 클래스 간 분산과 클래스 내부 분산의 비율을 최대화하는 방식으로 차원을 축소
⇒ 클래스간 분산을 최대한 크게, 내부 분산을 최소화
- PCA와 달리, 클래스 간 분산과 내부 분산 행렬을 생성한 뒤, 이 행렬에 기반해 고유 벡터를 구하고 입력데이터를 투영

LDA 과정

1. 클래스 내부와 클래스 간 분산 행렬을 구한다. 이 두개의 행렬은 입력 데이터의 결정 값 클래스별로 개별 피처의 평균 벡터를 기반으로 구함
2. 클래스 내부 분산 행렬을 S_w , 클래스 간 분산행렬을 S_B 라고 하면, 다음 식으로 두 행렬을 고유 벡터로 분해 가능

$$S_W^T S_B = \begin{bmatrix} e_1 & \cdots & e_n \end{bmatrix} \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

3. 고유값이 가장 큰 순으로 K개 추출
4. 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

4 SVD (Singular Value Decomposition)



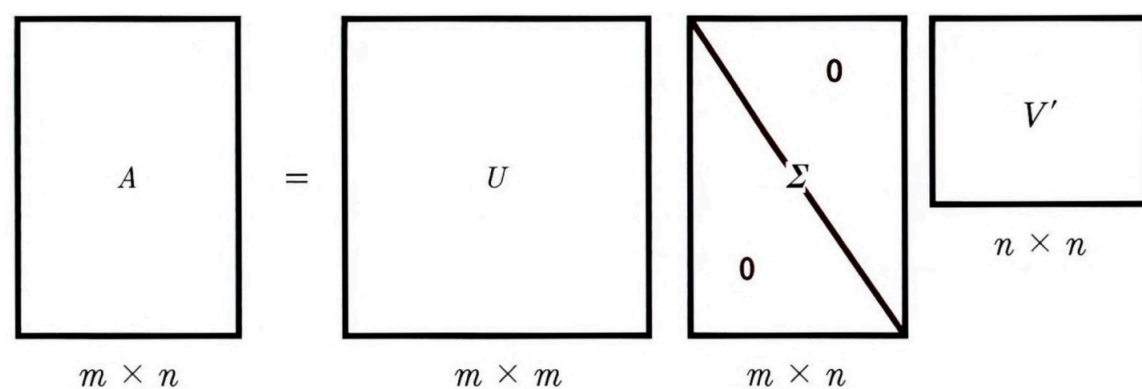
특이값 분해

PCA와 유사한 행렬 분해 기법을 이용하지만, 정방행렬 뿐 아니라, 행과 열의 크기가 다른 행렬에도 적용이 가능함

- 일반적으로 SVD는 $m \times n$ 크기의 행렬 A를 다음과 같이 분해

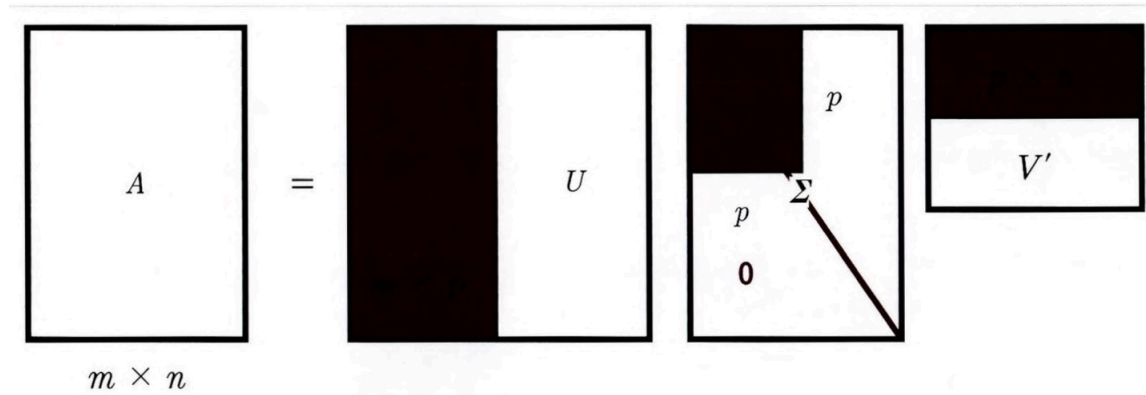
$$A = U \Sigma V^T$$

- 행렬 U와 V에 속한 벡터는 특이 벡터 (singular vector)
→ 모든 특이 벡터는 서로 직교하는 성질을 지님
- Σ 는 대각행렬, 행렬의 대각에 위치한 값만 0이 아니고 다른 값들이 모두 0
→ 이 행렬의 대각에 위치한 값이 바로 행렬 A의 특이값임
- SVD는 A의 차원이 $m \times n$ 일때 U의 차원이 $m \times m$, Σ 의 차원이 $m \times n$, V^T 의 차원이 $n \times n$ 로 분해



Truncated SVD

하지만 일반적으로는 대각 원소 중 특이값이 0인 부분도 모두 제거, 해당 위치에 대응하는 U와 V의 원소도 함께 제거해 차원을 줄인 형태로 적용함



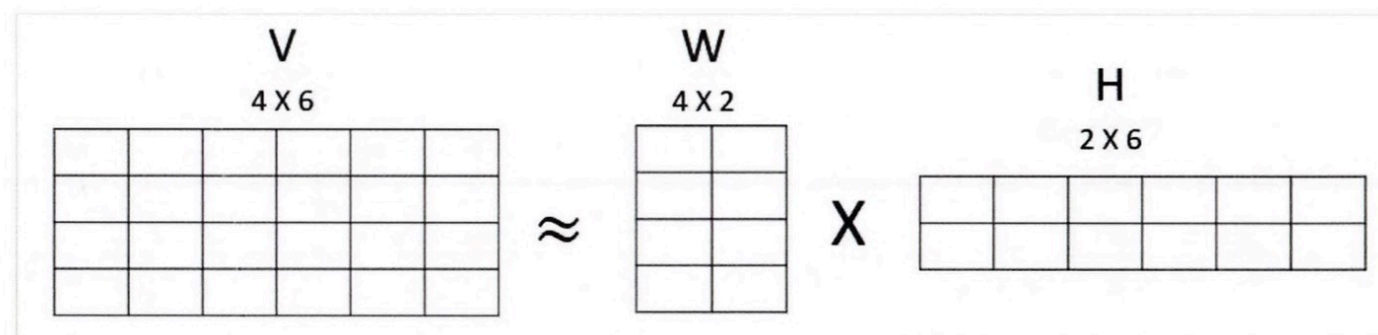
⇒ Σ 의 대각원소 중 상위 몇 개만 추출해서 여기에 대응하는 U와 V의 원소도 제거해 더욱 차원을 줄인 형태로 분해하는 것

- 인위적으로 더 작은 차원의 행렬로 분해하기 때문에 원본 행렬을 정확히 다시 원복할 수 없음
- 데이터 정보가 압축되어 분해됨에도 불구하고 상당한 수준으로 근사는 가능

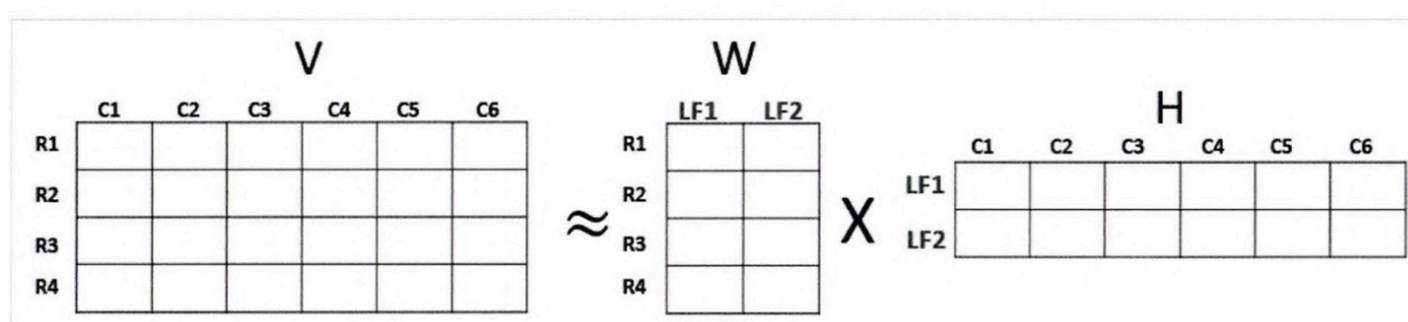
5 NMF (Non-Negative Matrix Factorization)

TruncatedSVD와 같이 낮은 랭크를 통한 행렬 근사 (Low-Rank Approximation) 방식의 변형

- 행렬 분해 (Matrix Factorization)
: 일반적으로 SVD와 같은 행렬 분해 기법을 통칭



- 원본 행렬 내의 모든 원소 값이 모두 0이상이라고 보장되면 두개의 기반 양수 행렬로 분해함
- 4x6 원본 행렬은 4x2 행렬 W와 2x6 행렬 H로 근사해 분해 가능



- 이렇게 분해된 행렬은 잠재 요소를 특성으로 가지게됨
 - 분해 행렬 W : 원본 행에 대해 이 잠재 요소의 값이 얼마나 되는지 대응
 - 분해 행렬 H : 원본 열(속성)로 잠재 요소가 어떻게 구성됐는지

⇒ SVD 와 유사하게 차원 축소를 통한 잠재 요소 도출로, 이미지 변환 및 압축, 텍스트의 토픽 도출 등의 영역에서 사용되고 있음

📌 정리

- 많은 피쳐로 이루어진 데이터 세트를 PCA 같은 차원 축소를 통해 더욱 직관적으로 이해할 수 있다
- 단순히 피쳐의 개수를 줄이기 보다는 이를 통해 데이터를 잘 설명할 수 있는 잠재적 요소를 추출 하는데 큰 의미가 있음
⇒ 많은 차원을 가지는 이미지, 텍스트에서 PCA, SVD 등의 차원 축소 알고리즘이 활발히 사용됨
- PCA

입력 데이터의 변동성이 가장 큰 축을 구하고, 다시 직교하는 축을 반복적으로 축소 차원 수만큼 구한 뒤, 입력 데이터를 투영해 차원을 축소

입력 데이터의 공분산 행렬 → 고유 벡터 생성 → 고유 벡터에 입력 데이터를 선형 변환

- LDA

PCA와 매우 유사, 다만 입력 데이터의 결정 값 클래스를 최대한 분리할 수 있는 축을 찾는 방식으로 축소

- SVD & NMF

매우 많은 피쳐 데이터를 가진 고차원 행렬을 두개의 저차원 행렬로 분리하는 기법

분해를 수행하며 잠재된 요소를 추출하기 때문에 토픽 모델링이나, 추천 시스템에서 활발하게 사용됨